



Airflow 3.0 Asset Watchers: Cross-Domain Data Mesh Orchestration with AI-Assisted Deployment

Haofei Feng

Senior Cloud Architect
Amazon Web Services

Corrine Tan

Cloud Architect
Amazon Web Services



Agenda

01 Challenges

Why enterprise demands federated Airflow

02 V2 Bridge

Reliability gaps in v2 and the patterns customers use today

03 Airflow 3

How Asset Watchers solve cross-account orchestration

04 Demo

walkthrough of Airflow 3 cross-account orchestration

05 Agent skill

Generate, validate, deploy, and verify DAGs with **agentic skills**

Enterprise scale requires federated Airflow ownership

CENTRALIZED MODEL

ONE AIRFLOW Across Organization

Simpler to provision, single control plane, lower initial cost, and unified monitoring.

Shared failure domain

Security boundary mismatch

Loss of control and flexibility



FEDERATED MODEL

CENTRAL DATA PLATFORM AWS ACCOUNT

Owns platform Airflow | Enforces data and security principles and guardrails

BU-A ACCOUNT

Owns domain
Airflow

BU-B ACCOUNT

Owns domain
Airflow

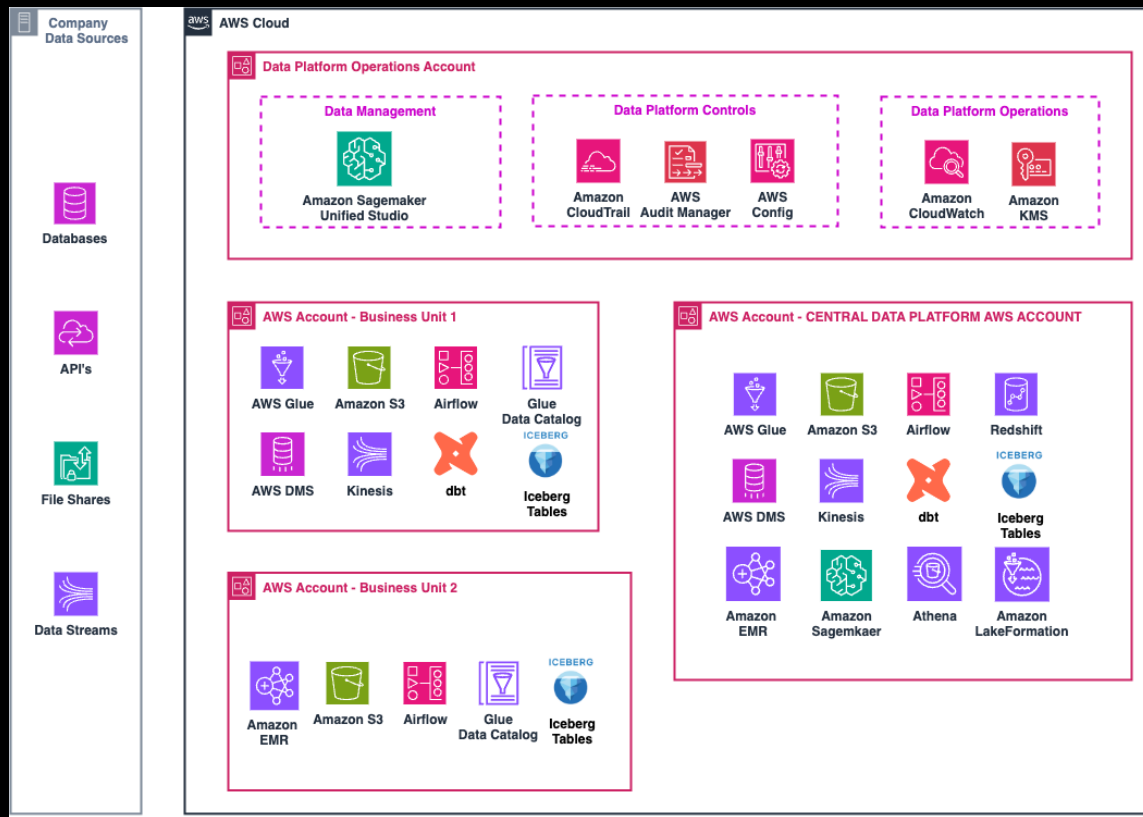
BU N ACCOUNT

Owns domain
Airflow

Team autonomy and account-level security isolation by design.

The federated data mesh: domain teams own their environment

- Domain-Driven Ownership
- Data as a product
- Federated computational governance
- Dedicated Airflow Instance per Domain
- Requires reliable orchestration across Airflow instances





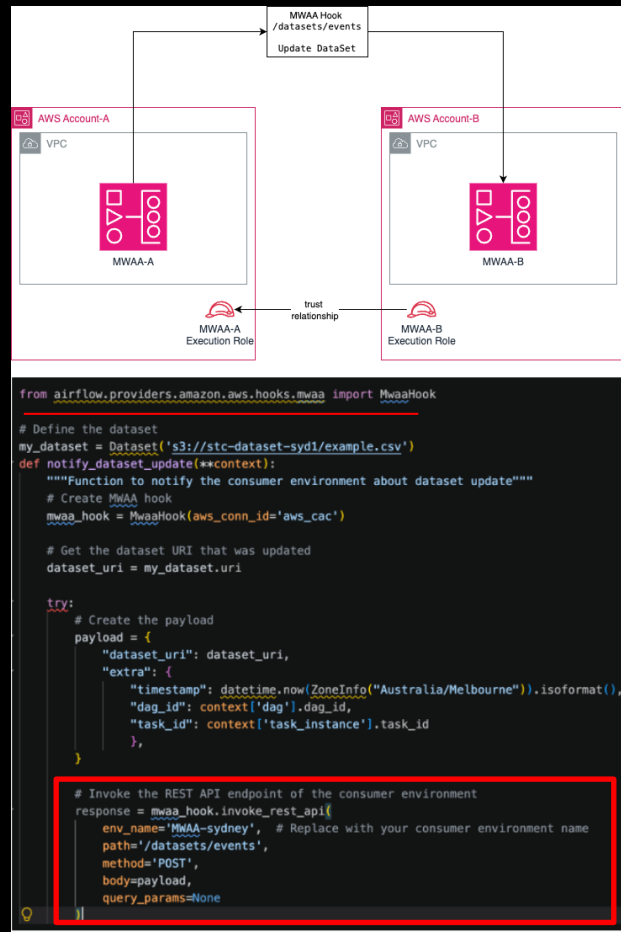
02 / Cross-instance orchestration in Airflow 2

How customers coordinate workflows across separate Airflow instances in Airflow 2



Airflow 2 Datasets: coordinating across instances with a REST hook

- Producer DAG finishes its work and fires a dataset event to the consumer instance
- The dataset is triggered and the consumer DAG runs straight away
- The MWAAHook (`airflow.providers.amazon.aws`) handles auth and calls the remote instance's `invoke_rest_api` to post the dataset event in one step



Reliability Issue: What breaks when you push dataset events across instances

1

Message Loss

5 messages fired within minutes, only 4 dataset events created. Consumer DAG runs 4 times instead of 5.

2

No Message Ordering

If we sent message in high frequency from producer side, the consumer was not able to process the events in the original order same with producer

3

No delivery guarantee

When the consumer DAG is **offline**, message are gone. Turning the DAG back on will not receive the messages

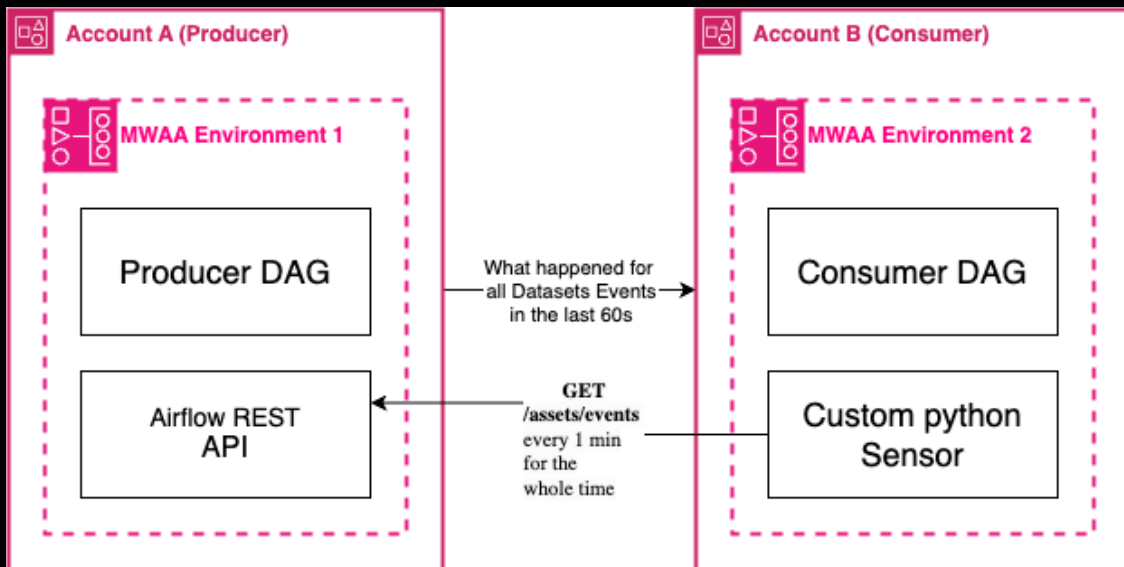
4

No observability

Can't prove 5 sent = 5 processed without custom code.

Workaround Solution

- Consumer pulls events from producer's REST API
- Custom code to improve reliability
- Costs a worker slot per dependency and adds polling latency





03 / Airflow 3 Asset Watchers



Timeline of Airflow Dataset and Asset

AIRFLOW 2.x • DATASET

2.4

Sep 2022

Datasets arrive

A task sends an event to a dataset, and a DAG can be scheduled on it.

2.9

Apr 2024

Conditions

Wait on several datasets at once – AND / OR.

2.10

Aug 2024

Aliases and extras

Name the dataset at runtime; attach metadata to the event.

AIRFLOW 3.x • ASSET

3.0

Apr 2025

Assets and watchers

Renamed to Asset – and an outside event can now update one.

3.2

Apr 2026

Partitions

One event updates one slice, not the whole asset.

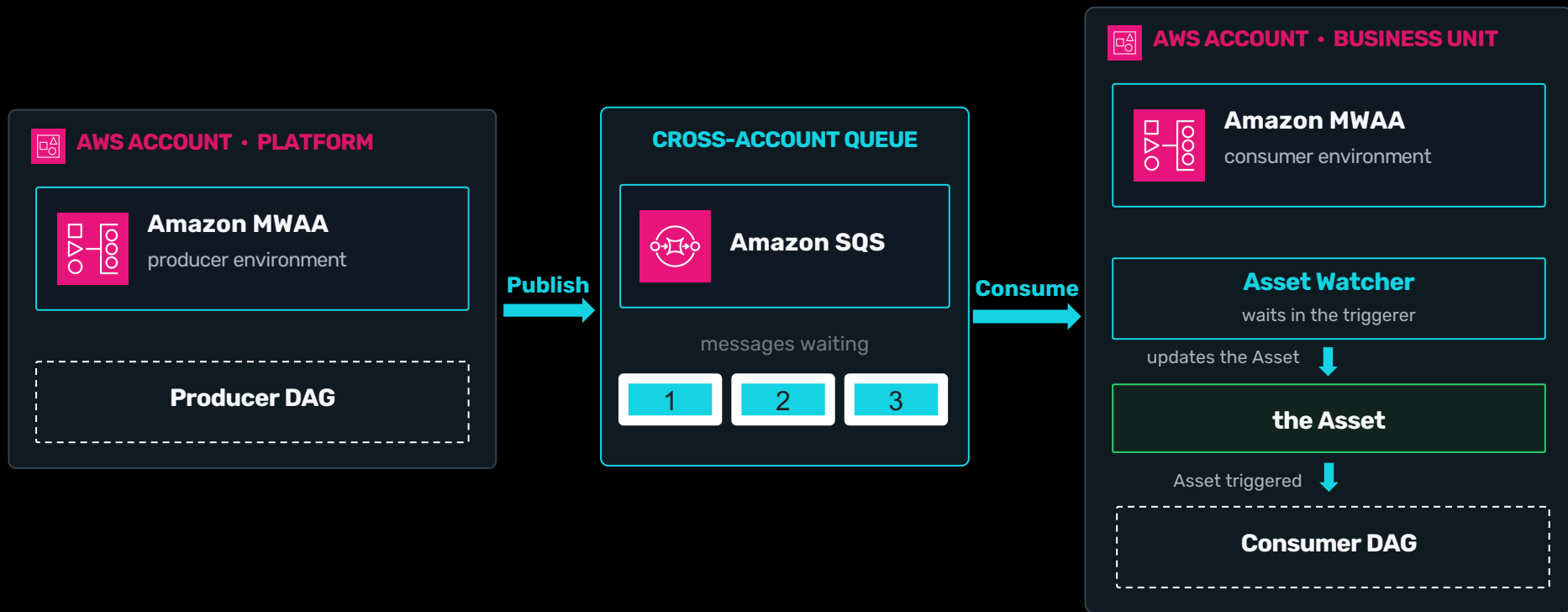
3.3

Jul 2026

Mappers

One event fans out to many; many roll up into one.

Our architecture after introducing Assets



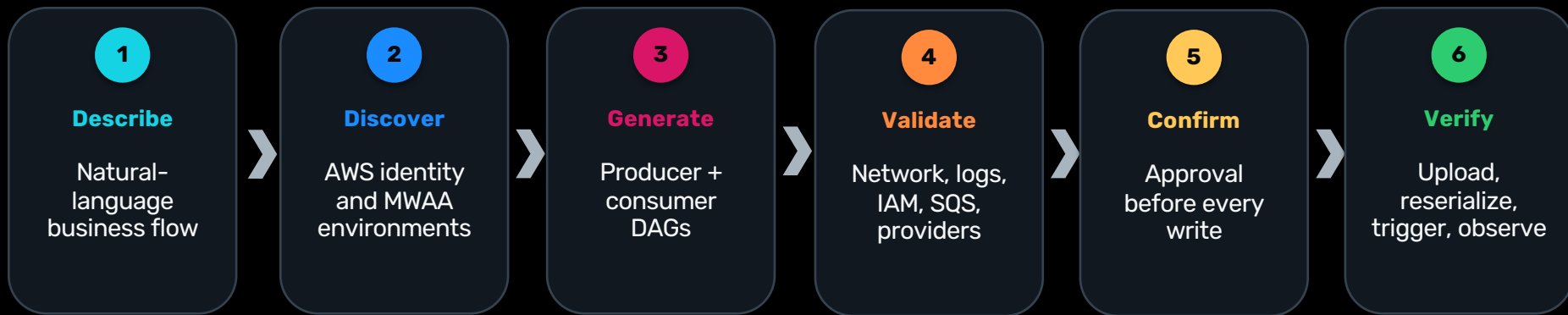
Neither environment can reach the other. Both can reach the queue.

04 / Agentic skill + demo

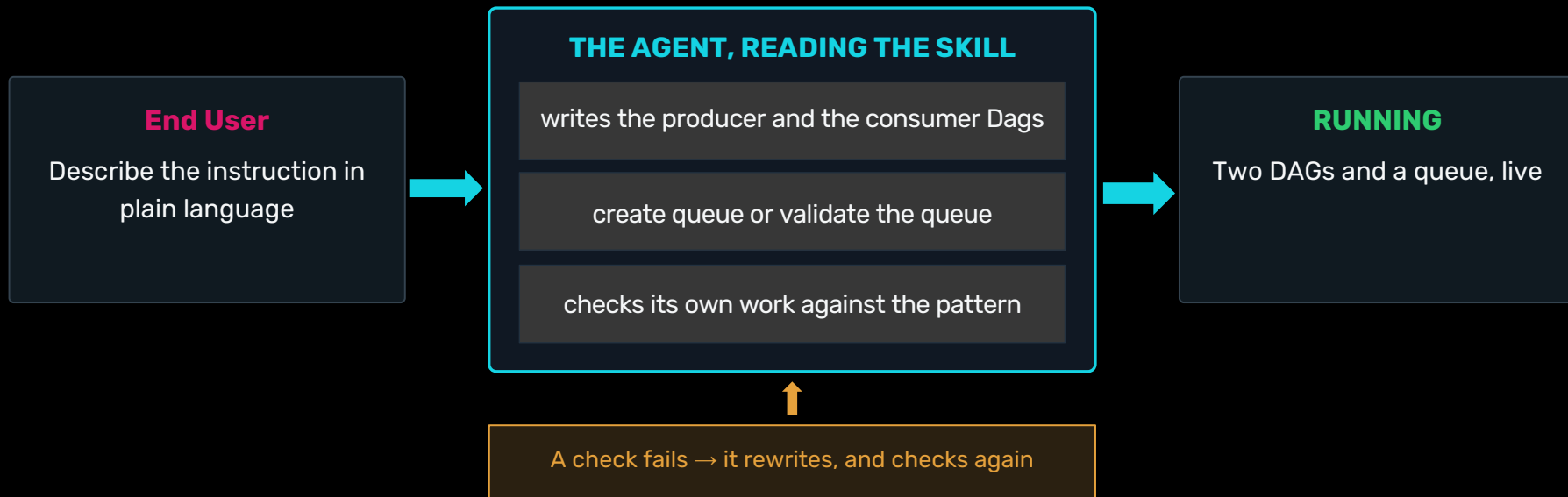


An agent skill, and the six steps it runs

A skill is a reusable package the agent reads before it acts: written instructions, the DAG templates, and the checks that have to pass.



Demo 1: One prompt, two DAGs, one queue



Demo 2: 5 events, pause in the middle and resume

WHAT I SEND

Five events, one by one

180 seconds apart

WHAT I BREAK

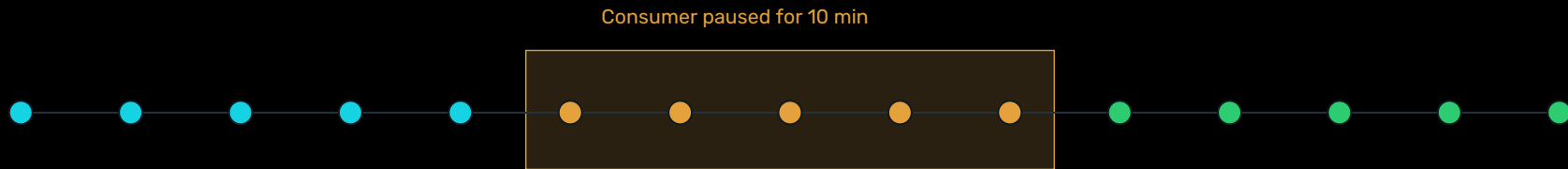
I stop the consumer halfway

Two minutes with nobody listening at all,
while producer keep sending events

WHAT SHOULD HAPPEN

Nothing is lost

It comes back and finishes the backlog:
each event once in order



What to watch: the backlog builds while nothing is listening: then every event lands, once each.

The four problems from Airflow 2 resolved

PROBLEM	HOW IT WAS TESTED	VERDICT
Message loss	non event lost, all events sent and received	Solved
Ordering	send and arrival order compared and matched	Queue choice (FIFO)
Delivery guarantee	consumer stopped for 8 min 23 s and resumed	Solved
Observability	SQS metrics provided the full picture	Solved

Nothing lost. Nothing applied twice. No manual intervention.



Questions?

Haofei Feng

Senior Cloud Architect

<https://www.linkedin.com/in/haofeifeng/>

Corrine Tan

Cloud Architect

<https://www.linkedin.com/in/corrine-tan12/>



AWS Blog: Event-driven pipeline orchestration
with Amazon MWAA and Airflow 3.0



Thank you! 🙏